

“DEVELOPMENT OF A SMART EMBEDDED AVIONICS AND TELEMETRY SYSTEM FOR EXPERIMENTAL ROCKETRY”

A PROJECT REPORT

Submitted to the PSG College of Arts & Science in partial fulfillment of the requirements
for the award of the degree of

MASTER OF SCIENCE IN APPLIED ELECTRONICS

Submitted by

SACHINDRA P (24MEL008)

Under the Guidance of

Dr. S. INDIRA M.Sc., M.Phil., Ph.D.,



DEPARTMENT OF ELECTRONICS

PSG COLLEGE OF ARTS & SCIENCE

An Autonomous College - Affiliated to Bharathiar University

Accredited with A⁺⁺ Grade by NAAC (4th cycle)

College with Potential for Excellence

(Status awarded by the UGC)

Star College Status awarded by DBT - MST

An ISO 9001:2015 Certified Institution

Coimbatore – 641 014.

APRIL 2026

CERTIFICATE

This is to certify that the project report entitled “**DEVELOPMENT OF A SMART EMBEDDED AVIONICS AND TELEMETRY SYSTEM FOR EXPERIMENTAL ROCKETRY** ” submitted to **PSG COLLEGE OF ARTS & SCIENCE** in partial fulfillment of the requirement for the award of the degree of “**MASTER OF SCIENCE IN APPLIED ELECTRONICS**” is a record of project work done by the **SACHINDRA P (24MEL008)** during the period of 2024-2026 in the Department of Electronics, PSG College of Arts and Science, Coimbatore – 641 014 under my supervision and guidance and the project report has not formed the basics for the award of any Degree/Diploma/Associateship/Fellowship or other similar title to any candidate of any university.

COUNTER SIGNED

Dr. P. Nandakumar M.Sc., M.Phil., Ph.D.,
HOD of Electronics [SF]
PSG College of Arts & Science
Coimbatore – 641 014.

SIGNATURE OF THE GUIDE

Dr. S. Indira M.Sc., M.Phil., Pd.D.,
Assistant Professor in Electronics
PSG College of Arts & Science
Coimbatore – 641 014.

Certified that the candidates were examined by us in the viva-voce examination held at
PSG College of Arts & Science, Coimbatore.

held on _____

Internal Examiner

External Examiner

DECLARATION

I **SACHINDRA P (24MEL008)** declare that the project report entitled **“DEVELOPMENT OF A SMART EMBEDDED AVIONICS AND TELEMETRY SYSTEM FOR EXPERIMENTAL ROCKETRY ”** submitted to PSG College of Arts & Science, Coimbatore in the partial fulfillment of the requirements for the award of the degree of **“MASTER OF SCIENCE IN APPLIED ELECTRONICS”** is a report of the project work done by me under the supervision and guidance of **Dr. S. INDIRA M.Sc., M.Phil., Ph.D., Assistant Professor, Department of Electronics, PSG College of Arts & Science, Coimbatore – 641014.**

SACHINDRA P (24MEL008)

ACKNOWLEDGEMENT

I express my sincere gratitude to my esteemed guide, **Dr. S. Indira, M.Sc., M.Phil., Ph.D., Assistant Professor, Department of Electronics, PSG College of Arts & Science, Coimbatore**, for her invaluable guidance, consistent encouragement, and thoughtful suggestions throughout the course of this project.

I am deeply grateful to **Dr. P. Nandakumar, M.Sc., MCA., M.Phil., Ph.D., HOD, Department of Electronics, PSG College of Arts & Science**, for his constant support, insightful advice, and academic leadership which greatly contributed to the successful completion of my work.

I extend my heartfelt thanks to our respected Principal, **Dr. M. Jayanthi, M.Com., M.Phil., Ph.D.** for providing all the necessary facilities and a conducive environment for carrying out my project efficiently.

I also wish to thank our honourable Secretary, **Dr. T. Kannaian, M.Tech., Ph.D.**, for offering excellent infrastructure and continuous support in fostering innovation and technical development.

Finally, I convey my deep appreciation to my parents, friends, and all the staff members of the Department of Electronics, PSG College of Arts & Science, Coimbatore, for their moral support, encouragement, and timely help during every phase of my project journey.

ABSTRACT

IRIS is an experimental aerospace project focused on the design and development of an intelligent rocket avionics, control, and telemetry system for real-time flight monitoring. The primary objective is to develop a compact, reliable, and cost-effective onboard flight computer capable of acquiring, processing, and transmitting critical flight parameters during a rocket mission using interdisciplinary engineering techniques.

The system architecture is centred on an ESP32 microcontroller, selected for its high computational capability, low power consumption, and integrated wireless communication features. The flight computer interfaces with multiple sensors for accurate data acquisition. A BMP280 barometric pressure sensor is used for altitude and atmospheric measurements, while an MPU6050 inertial measurement unit provides acceleration and attitude data. A NEO-6 GPS module supplies real-time positional information.

Wireless telemetry is implemented using an NRF24L01 long-range communication module, enabling continuous transmission of flight data to a ground control station for real-time monitoring and post-flight analysis. The control subsystem employs servo-actuated aerodynamic mechanisms to improve flight stability, while an emergency recovery mechanism enhances overall system safety and reliability.

The IRIS project demonstrates that advanced rocket avionics systems can be effectively developed through embedded systems engineering, sensor integration, control electronics, and wireless communication, contributing to modern experimental rocketry and future avionics research.

CONTENTS

CHAPTER	TOPICS	PAGE NO.
1.	1.1 INTRODUCTION	1
2.	2.1 BLOCK DIAGRAM	2
	2.2 ESP32 (38Pin) MICROCONTROLLER	3
	2.3 NODE MCU CP2102	3
	2.4 BMP280 SENSOR	3
	2.5 MPU6050 IMU SENSOR	4
	2.6 NEO6M GPS MODULE	4
	2.7 NRF24L01 LONG RANGE (Wi-Fi)	4
	2.8 SG90 SERVO MOTOR	5
	2.9 MP1584 BUCK CONVERTER	5
	2.10 LITHIUM-ION BATTERY	5
3.	3.1 CIRCUIT DIAGRAM	6
4.	HARDWARE	
	4.1 ESP32 (38Pin) MICROCONTROLLER	8
	4.2 NODE MCU CP2102	12
	4.3 BMP280 SENSOR	15
	4.4 MPU6050 IMU SENSOR	17
	4.5 NEO6M GPS MODULE	19
	4.6 NRF24L01 LONG RANGE (Wi-Fi)	21
	4.7 SG90 SERVO MOTOR	23
	4.8 MP1584 BUCK CONVERTER	25
	4.9 LITHIUM-ION BATTERY	26

5.	SOFTWARE	
5.1	ARDUINO IDE	28
5.2	COSMIC TOWER V1.2.0	31
5.3	FLOW CHART	34
5.4	PROGRAM	35
6.	6.1 TESTING	51
7.	7.1 CONCLUSION	57
8.	BIBLIOGRAPHY	58

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

The rapid advancement of aerospace technology has significantly transformed the way space missions are conducted, with a strong focus on reusability, cost efficiency, and autonomous control systems. One of the most revolutionary developments in modern rocketry is the concept of self-landing rockets, as demonstrated by advanced aerospace systems. These systems reduce mission costs and improve operational efficiency by enabling the recovery and reuse of launch vehicles.

This project, titled “IRIS Aerospace Self-Landing Rocket Prototype,” aims to design and develop a scaled-down experimental rocket system capable of autonomous stabilization, controlled flight, and safe recovery. The prototype integrates embedded systems, sensor fusion, wireless telemetry, and real-time control mechanisms to simulate the working principles of modern reusable rockets.

The system is built around an ESP32-based flight computer, which collects real-time data from sensors such as BMP280 (altitude, pressure, temperature) and MPU6050 (acceleration and orientation). This data is processed using control algorithms to maintain flight stability through Thrust Vector Control (TVC) and aerodynamic fin control systems. The rocket also includes a wireless communication system (NRF24L01) to transmit telemetry data to a Ground Control Station (GCS) for real-time monitoring and command execution.

To ensure safety and reliability, the system incorporates features such as pre-flight calibration, fail-safe mechanisms, and parachute deployment for emergency recovery. The project also emphasizes real-time data visualization and analysis using Cosmic Tower Aerospace Software, enabling clear graphical representation and better understanding of flight parameters and dynamics.

Overall, this project serves as a practical implementation of aerospace engineering concepts, embedded systems design, and control theory. It provides a foundational platform for developing advanced reusable rocket technologies and contributes to learning in the fields of avionics, telemetry, and autonomous systems.

CHAPTER 2

BLOCK DIAGRAM

2.1 BLOCK DIAGRAM

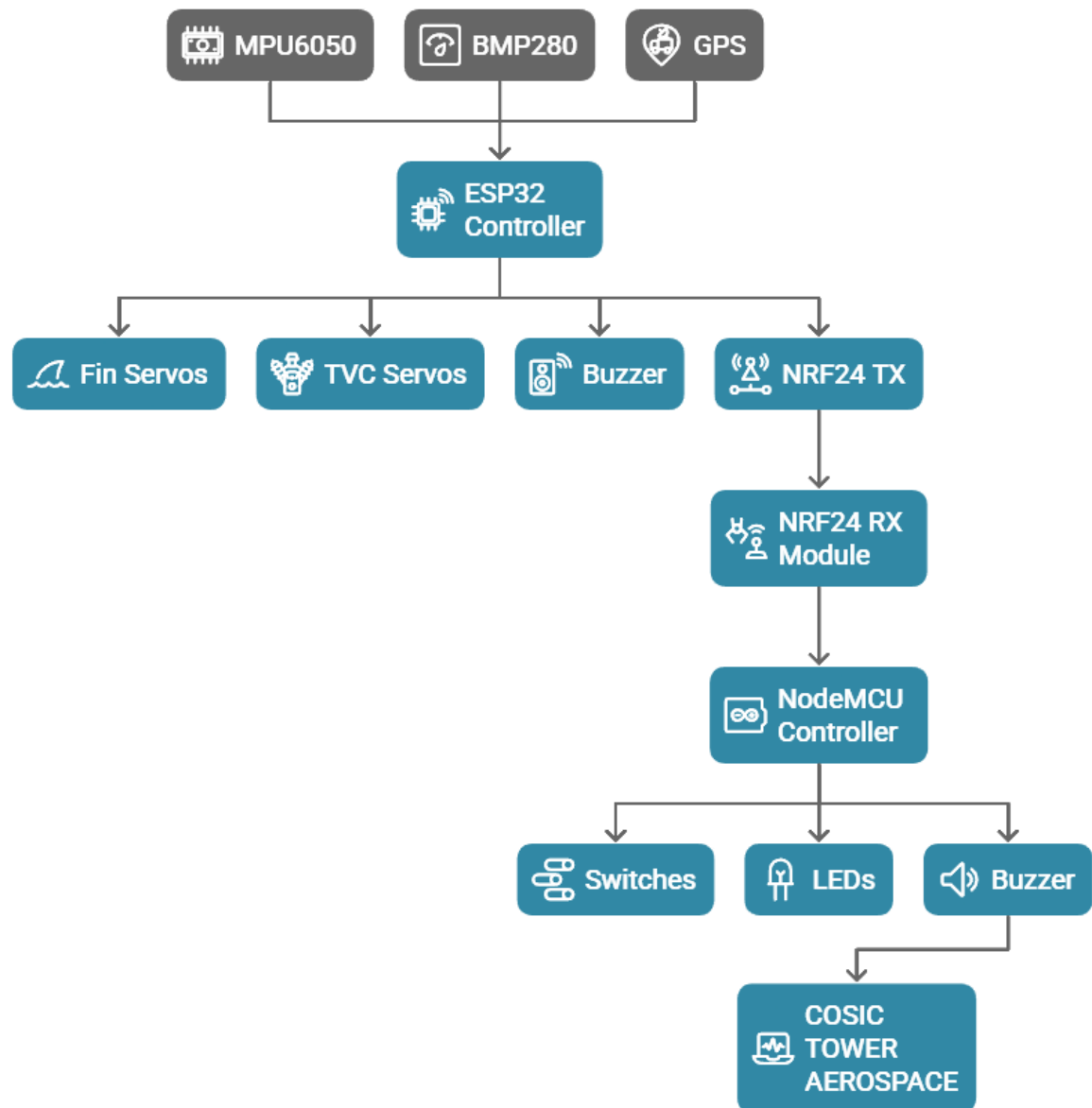


Figure 2: Block Diagram

2.2 ESP32 (38-Pin) MICROCONTROLLER

The ESP32 is a powerful and versatile microcontroller widely used in embedded systems and IoT applications due to its high performance and integrated features. It is equipped with a dual-core processor operating at up to 240 MHz, along with built-in Wi-Fi and Bluetooth capabilities, making it suitable for wireless communication. The ESP32 supports multiple communication protocols such as I2C, SPI, and UART, allowing easy interfacing with various sensors and modules. In the IRIS Aerospace Rocket System, the ESP32 acts as the main flight computer, responsible for processing sensor data, controlling actuators, and managing telemetry communication. Its low power consumption and high processing capability make it ideal for real-time aerospace applications.

2.3 NODE MCU CP2102

The NodeMCU CP2102 is a Wi-Fi-enabled microcontroller based on the ESP8266, widely used for IoT applications. It includes a CP2102 USB-to-Serial converter for easy programming and communication. The board supports UART, SPI, and I2C protocols for interfacing with external devices. In the IRIS Aerospace Rocket System, it is used as the Ground Control Station (GCS) receiver to collect telemetry data and transmit it for monitoring. Its compact design and wireless capability make it suitable for real-time communication systems.

2.4 BMP280

The BMP280 is a high-precision digital barometric pressure sensor used for measuring atmospheric pressure, temperature, and altitude. It is widely used in weather monitoring and altitude measurement applications. The sensor provides accurate readings with low power consumption and supports I2C and SPI communication interfaces. In the IRIS Aerospace Rocket System, the BMP280 is used to measure the altitude of the rocket relative to the launch point. This data is essential for flight monitoring, control decisions, and parachute deployment. Its compact size and reliability make it suitable for embedded aerospace systems.

2.5 MPU6050 IMU SENSOR

The MPU6050 is a 6-axis Inertial Measurement Unit (IMU) that combines a 3-axis accelerometer and a 3-axis gyroscope in a single module. It is used to measure acceleration, angular velocity, and orientation of the system. The sensor communicates with the microcontroller using the I2C protocol and provides real-time motion data. In the IRIS Aerospace Rocket System, the MPU6050 plays a crucial role in detecting the rocket's tilt, rotation, and movement during flight. This information is used for stabilization through thrust vector control and grid fin adjustments, ensuring controlled and stable flight behavior.

2.6 NEO-6M GPS MODULE

The NEO-6M GPS module is a satellite-based positioning system used to determine the exact geographical location of the rocket. It receives signals from GPS satellites and provides data such as latitude, longitude, and altitude. The module communicates with the ESP32 using UART communication. In the IRIS Aerospace Rocket System, the GPS module is used for tracking the rocket's position during flight and recovery. It ensures accurate location monitoring, which is useful for navigation, analysis, and retrieval of the rocket after landing.

2.7 NRF24L01 LONG RANGE (Wi-Fi)

The NRF24L01 is a wireless transceiver module used for communication between the rocket and the ground control station. It operates in the 2.4 GHz frequency band and provides reliable long-range communication with low power consumption. The module uses SPI communication for interfacing with the ESP32. In the IRIS Aerospace Rocket System, the NRF24L01 is used to transmit telemetry data such as altitude, sensor readings, and flight status to the ground station, and also to receive control commands. This enables real-time monitoring and bidirectional communication during flight.

2.8 SG90 SERVO MOTOR

The SG90 servo motor is a compact and lightweight actuator used for precise angular control in embedded systems. It operates based on Pulse Width Modulation (PWM)

signals, which allow it to rotate to specific angles typically ranging from 0° to 180°. The SG90 servo is widely used due to its low power consumption, fast response, and accurate positioning capabilities. In the IRIS Aerospace Rocket System, the SG90 servo motors are used to control the grid fins and thrust vector control (TVC) mechanism, which help in maintaining the stability and direction of the rocket during flight. The ESP32 microcontroller generates PWM signals to control the movement of the servos based on real-time sensor data. This enables precise orientation control and improves the overall performance of the rocket system

2.9 MP1584 BUCK CONVERTER

The MP1584 is a high-efficiency DC-DC buck converter used to step down voltage from a higher level to a stable lower voltage. It provides a regulated output with minimal power loss and high efficiency. In the IRIS Aerospace Rocket System, the MP1584 is used to supply stable voltage to the ESP32, sensors, and communication modules. Proper voltage regulation is essential for the reliable operation of electronic components, especially in aerospace systems where power stability directly affects performance and safety.

2.10 LITHIUM-ION BATTERY

The Lithium-Ion battery is a rechargeable energy storage device widely used in portable electronic systems due to its high energy density and lightweight design. It provides a stable and reliable power source for electronic components. In the IRIS Aerospace Rocket System, it is used to supply power to the flight computer, sensors, communication modules, and actuators. Its long-life cycle and efficient performance make it suitable for aerospace and embedded applications.

CHAPTER 3

CIRCUIT DIAGRAM

3.1 CIRCUIT DIAGRAM

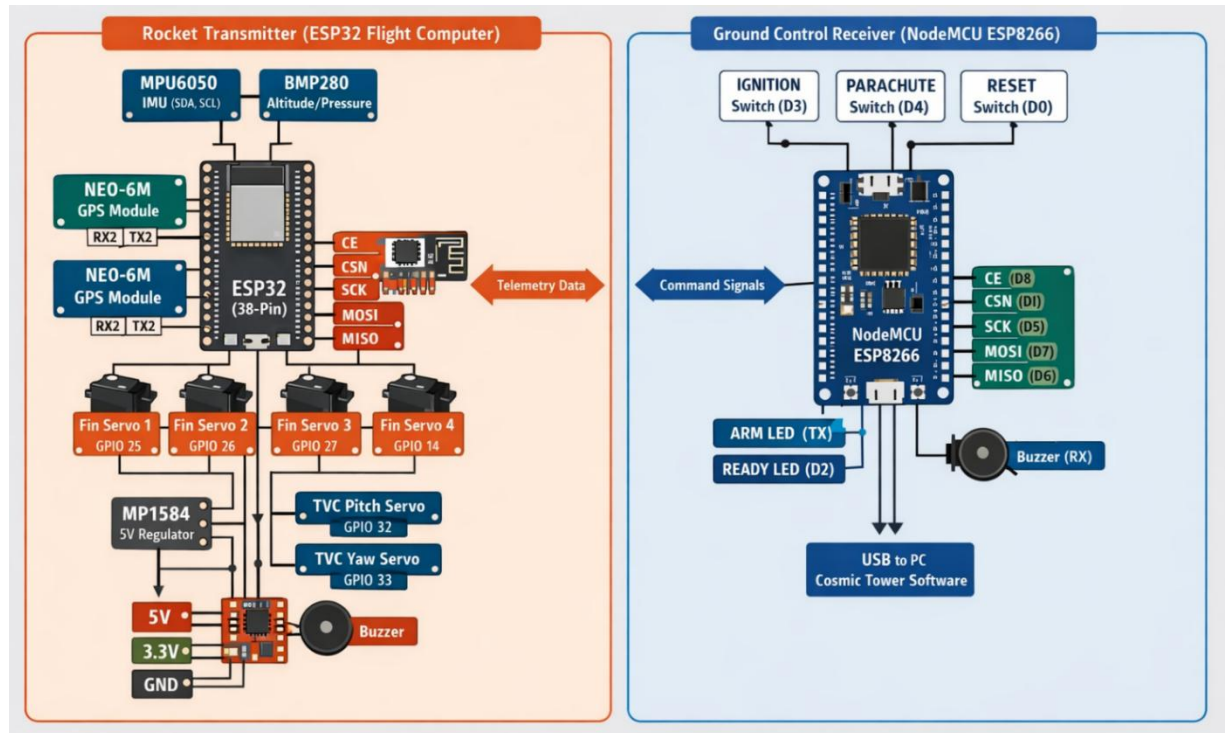


Figure 2: Circuit Diagram

The Figure 2 represents the circuit diagram of the IRIS Aerospace Rocket System, which illustrates the interconnection of various electronic components used in the system. The entire project operates through a series of interconnected modules, each contributing to real-time monitoring, control, communication, and safety of the rocket system.

The system begins by continuously collecting data from onboard sensors such as the MPU6050 IMU sensor, BMP280 barometric sensor, and GPS module. These sensors provide real-time information about acceleration, orientation, altitude, pressure, temperature, and position of the rocket. The MPU6050 measures motion and tilt, while the BMP280 calculates altitude based on atmospheric pressure. The GPS module provides accurate location tracking.

The sensor data is processed by the ESP32 microcontroller, which acts as the central flight computer. The ESP32 reads and analyzes all incoming data, performs calibration and filtering, and converts it into meaningful information required for flight control. Based on this data, the system determines the rocket's orientation and stability during flight.

The processed data is then transmitted using the NRF24L01 wireless module, which sends telemetry information to the ground control station. The ground station receives this data for real-time monitoring and can also send control commands such as ignition, reset, or parachute deployment back to the rocket.

The ESP32 also controls actuator components such as thrust vector control (TVC) servos and grid fin servos, which help in maintaining stability and controlling the direction of the rocket. These components receive PWM signals from the controller to adjust the rocket's movement during flight.

In addition, the system includes an ignition control circuit for launching the rocket and a parachute deployment mechanism for safe landing. These are activated based on predefined conditions or through commands from the ground station.

The entire system is powered using a regulated power supply provided by the MP1584 buck converter, which ensures stable voltage distribution to all components including sensors, communication modules, and actuators.

Overall, the circuit diagram clearly shows how different components are electrically connected and work together as a complete system. This integration enables efficient data acquisition, processing, communication, and control, ensuring reliable performance of the IRIS Aerospace Rocket System

CHAPTER 4

HARDWARE COMPONENTS

4.1 ESP32 WROOM MODULE

The Figure 3 represents ESP32 WROOM (38-Pin) Microcontroller, which is a development board based on the ESP32 system-on-chip (SoC) manufactured by Espressif Systems. The ESP32 is a widely used microcontroller that offers both Wi-Fi and Bluetooth connectivity, making it suitable for a wide range of IoT (Internet of Things) and embedded system applications.

ESP32 SoC: The ESP32 is a dual-core microcontroller with a 32-bit CPU architecture. It features integrated Wi-Fi (802.11 b/g/n) and Bluetooth (BLE) capabilities. It also supports multiple communication interfaces such as GPIO, UART, SPI, I2C, ADC, and PWM, making it highly versatile for various applications.

Development Board: The ESP32 Dev Kit is designed for easy prototyping and development. It includes onboard voltage regulation, accessible GPIO pins, and a USB-to-Serial converter for programming and debugging.

Programming: The ESP32 can be programmed using Arduino IDE, Platform IO, and Espressif's ESP-IDF framework. It supports C and C++ programming languages.

Power Supply: The board can be powered using a USB connection or an external power supply, typically ranging from 5V to 12V depending on the application.

I/O Pins: The ESP32 provides multiple GPIO pins that can be used to connect sensors, actuators, communication modules, and other peripherals.

Wi-Fi and Bluetooth: The ESP32 supports wireless communication through built-in Wi-Fi and Bluetooth, allowing real-time data transmission and remote-control applications.

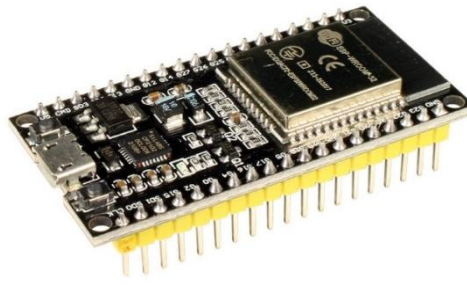


Figure 3: ESP32 WROOM Module

Pin Description:

Common GPIO Pins (General-Purpose Digital I/O)

GPIO0: Boot mode selection, general-purpose digital I/O

GPIO2: Bootstrapping pin, general-purpose digital I/O

GPIO4–GPIO14: General-purpose digital I/O

GPIO15: Boot mode selection, general-purpose digital I/O

GPIO16, GPIO17: General-purpose digital I/O

GPIO18: General-purpose digital I/O, often used as SPI clock (SCK)

GPIO19: General-purpose digital I/O, often used as SPI MISO

GPIO21: General-purpose digital I/O, often used as I2C SDA

GPIO22: General-purpose digital I/O, often used as I2C SCL

GPIO23: General-purpose digital I/O, often used as SPI MOSI

GPIO25, GPIO26, GPIO27: General-purpose digital I/O

I2C Pins (SDA and SCL):

- **SDA (Serial Data)** (Commonly GPIO21): This pin is used for the data line in I2C communication. It is used for sending and receiving data between the ESP32 and I2C devices.
- **SCL (Serial Clock)** (Commonly GPIO22): This pin is used for the clock line in I2C communication, which synchronizes data transfer between devices.

UART Pins (TX and RX):

- **TX (Transmit)** (Commonly GPIO1): This pin is used for transmitting data from the ESP32 to an external UART device.

- **RX (Receive)** (Commonly GPIO3): This pin is used for receiving data from an external UART device.

SPI Pins (MOSI, MISO, SCK, and CS):

- **MOSI (Master Out Slave In)** (Commonly GPIO23): MOSI is the data line for sending data from ESP32 to SPI devices.
- **MISO (Master In Slave Out)** (Commonly GPIO19): MISO is the data line for receiving data from SPI devices.
- **SCK (Serial Clock)** (Commonly GPIO18): SCK is the clock line used for synchronizing data transfer in SPI communication.
- **CS (Chip Select)** (Commonly GPIO5): The Chip Select Pin is used to select a specific SPI device when multiple devices share the same SPI bus. Note that the CS pin assignment can vary depending on the board.

ADC Pins

Common ADC pins include GPIO32 to GPIO39. These pins are often labeled as ADC1_CH0 to ADC1_CH7, corresponding to channels 0 to 7. They are used for analog-to-digital conversion and can read analog voltage levels from sensors and other analog sources.

PWM Pins

Various GPIO pins on the ESP32 support PWM (Pulse Width Modulation) output. Common PWM-capable GPIO pins include: GPIO 0, GPIO 2, GPIO 4, GPIO 5, GPIO 12, GPIO 13, GPIO 14, GPIO 15, GPIO 16, GPIO 17, GPIO 18, GPIO 19, GPIO 21, and GPIO 22. These pins can generate PWM signals and are often used for controlling the brightness of LEDs, motor speed, and other applications that require variable output levels.

Enable Pin

The Enable (EN) pin is typically not a GPIO pin, and it may not be directly accessible as a GPIO. It is used to enable or disable the ESP32 module and control its power state. The exact implementation can vary between boards.

Boot Mode Pins

The boot mode pins are GPIO 0 and GPIO 2. Their configuration during startup determines the boot mode of the ESP32. GPIO 0 is commonly used for this purpose.

Interrupt-Capable GPIO Pins

GPIO0: Boot mode selection, general-purpose digital I/O, external interrupt input

GPIO2: Bootstrapping, general-purpose digital I/O, external interrupt input

GPIO4: General-purpose digital I/O, external interrupt input

GPIO5: General-purpose digital I/O, external interrupt input

GPIO12–GPIO15: General-purpose digital I/O, external interrupt input

Specifications

- **Microcontroller:** Dual-core Tensilica LX6 processor
- **CPU Frequency:** Adjustable up to 240 MHz
- **Operating Voltage:** 2.2V to 3.6V
- **Wireless Connectivity:**
 - Wi-Fi: 802.11 b/g/n (2.4 GHz)
 - Bluetooth: Bluetooth Classic (BT 4.2 BR/EDR) and BLE
- **Memory:**
 - SRAM: 520 KB
 - Flash: 4 MB
- **GPIO Pins:** Multiple GPIO pins for various interfaces
- **Analog-to-Digital Converter (ADC):** 12-bit SAR ADC
- **Interfaces:** UART, SPI, I2C, PWM, CAN, SDIO
- **Power Consumption:** Ultra-low power consumption in sleep modes
- **Operating Temperature:** -40°C to 85°C
- **Physical Specifications (for a typical ESP32 Dev Kit):**
- **Dimensions:** Approximately 100 mm × 60 mm
- **Power Supply:** USB or external 5V to 12V power source
- **USB-to-Serial Converter:** Onboard USB-to-UART bridge for programming and debugging

Programming and Development:

- **Development Environment:** Arduino IDE, platform IO, Espressif's ESP-IDF (IoT Development Framework), and more
- **Programming Language:** C/C++
- **Programming Interfaces:** Supports flashing and debugging via USB
- **Firmware Updates:** Over-the-air (OTA) firmware updates supported

Wireless Connectivity:

- **Wi-Fi:** Supports 802.11 b/g/n with WPA/WPA2, WEP, and WPS security
- **Bluetooth:** Supports Bluetooth Classic (BT 4.2 BR/EDR) and Bluetooth Low Energy (BLE)

4.2 NODEMCU CP2102 (ESP8266) MICROCONTROLLER

The Figure 4 Represents NodeMCU CP2102 is a development board based on the ESP8266 microcontroller, widely used for IoT and wireless communication applications. It features an integrated Wi-Fi module that allows devices to connect to the internet and perform real-time data transmission. The CP2102 USB-to-Serial converter enables easy programming and debugging through a USB interface.

The NodeMCU is commonly used as a communication controller in embedded systems. In the IRIS Aerospace Rocket System, it is used in the Ground Control Station (GCS) to receive telemetry data from the rocket and send control commands such as ignition, parachute deployment, and system reset.



Figure 4: NodeMCU CP2102

Pin Description

Common GPIO Pins (General-Purpose Digital I/O)

GPIO0: Boot mode selection, general-purpose digital I/O

GPIO1: TX (Transmit), general-purpose digital I/O

GPIO2: Bootstrapping pin, general-purpose digital I/O

GPIO3: RX (Receive), general-purpose digital I/O

GPIO4: General-purpose digital I/O

GPIO5: General-purpose digital I/O

GPIO12: General-purpose digital I/O, often used as SPI MISO

GPIO13: General-purpose digital I/O, often used as SPI MOSI

GPIO14: General-purpose digital I/O, often used as SPI clock (SCK)

GPIO15: Boot mode selection, general-purpose digital I/O

GPIO16: General-purpose digital I/O

I2C Pins (SDA and SCL):

- **SDA (Serial Data) (Commonly GPIO4):** This pin is used for the data line in I2C communication. It is used for sending and receiving data between the NodeMCU and I2C devices.
- **SCL (Serial Clock) (Commonly GPIO5):** This pin is used for the clock line in I2C communication, which synchronizes data transfer between devices.

UART Pins (TX and RX):

- **TX (Transmit) (Commonly GPIO1):** This pin is used for transmitting data from the NodeMCU to an external UART device.
- **RX (Receive) (Commonly GPIO3):** This pin is used for receiving data from an external UART device.

SPI Pins (MOSI, MISO, SCK, and CS):

- **MOSI (Master Out Slave In) (Commonly GPIO13):** MOSI is the data line for sending data from NodeMCU to SPI devices.
- **MISO (Master In Slave Out) (Commonly GPIO12):** MISO is the data line for receiving data from SPI devices.
- **SCK (Serial Clock) (Commonly GPIO14):** SCK is the clock line used for synchronizing data transfer in SPI communication.

- **CS (Chip Select) (Commonly GPIO15):** The Chip Select Pin is used to select a specific SPI device when multiple devices share the same SPI bus.

ADC Pins:

- **A0:** Single analog input pin available on NodeMCU
- Input voltage range: 0V to 1V (internally scaled for higher voltages)
- Used for analog-to-digital conversion and reading sensor values

PWM Pins

- Most GPIO pins support PWM (Pulse Width Modulation) output
- Common PWM pins include: GPIO0, GPIO2, GPIO4, GPIO5, GPIO12, GPIO13, GPIO14, GPIO15
- These pins can generate PWM signals and are used for LED brightness control, motor speed control, etc.

Enable Pin

- **EN (CH_PD):** The Enable pin is used to activate the ESP8266 module
- Must be HIGH for normal operation
- Controls power state of the module

Boot Mode Pins

- GPIO0, GPIO2, and GPIO15 determine boot mode
- GPIO0 LOW → Programming mode
- GPIO0 HIGH → Normal execution
- GPIO15 must be LOW during boot

Interrupt-Capable GPIO Pins

- **GPIO0:** Boot mode selection, general-purpose I/O, external interrupt input
- **GPIO2:** Bootstrapping, general-purpose digital I/O, external interrupt input
- **GPIO4:** General-purpose digital I/O, external interrupt input
- **GPIO5:** General-purpose digital I/O, external interrupt input
- **GPIO12–GPIO15:** General-purpose digital I/O, external interrupt input

Specifications

- **Microcontroller:** ESP8266
- **CPU Frequency:** 80 MHz (up to 160 MHz)
- **Operating Voltage:** 3.0V to 3.6V

- **Wireless Connectivity:**
 - ▶ Wi-Fi: 802.11 b/g/n (2.4 GHz)
- **Memory:**
 - ▶ SRAM: 96 KB
 - ▶ Flash: 4 MB
- **GPIO Pins:** Multiple digital I/O pins
- **ADC:** 10-bit ADC
- **Interfaces:** UART, SPI, I2C (software), PWM
- **Power Consumption:** Low power with deep sleep mode
- **Operating Temperature:** -40°C to 125°C

Physical Specifications (for a typical NodeMCU CP2102 Board):

- **Dimensions:** Approximately 58 mm × 31 mm
- **Power Supply:** USB (5V) or external 3.3V
- **USB-to-Serial Converter:** CP2102

Wireless Connectivity:

- **Wi-Fi:** Supports TCP/IP stack
- **Modes:** Station (STA), Access Point (AP), or both

4.3 BMP280 SENSOR

A figure 5 Represents BMP280 Sensor, which is a precision environmental sensor developed by Bosch. It is used to measure atmospheric pressure and temperature with high accuracy. The BMP280 is widely used in embedded systems, IoT applications, weather monitoring systems, and altitude measurement systems.

The sensor operates using a piezoresistive pressure sensing element and provides highly accurate readings with low power consumption. It communicates with microcontrollers like ESP32 using digital communication protocols such as I2C and SPI. The BMP280 is suitable for applications where accurate pressure and temperature measurements are required, such as altitude estimation, environmental monitoring, and weather forecasting systems.



Figure 5: BMP280 Sensor

Pin Description

Common Pins:

VCC: Power supply input (1.8V to 3.6V)

GND: Ground connection

SDA: Serial Data line for I2C communication

SCL: Serial Clock line for I2C communication

CSB: Chip Select pin for SPI communication

SDO: Serial Data Output (used in SPI and I2C address selection)

I2C Pins (SDA and SCL):

- **SDA (Serial Data):** Used for transmitting and receiving data between BMP280 and ESP32.
- **SCL (Serial Clock):** Used to synchronize communication between devices.

SPI Pins (MOSI, MISO, SCK, and CS):

- **MOSI:** Master Out Slave In (Data from ESP32 to BMP280)
- **MISO:** Master In Slave Out (Data from BMP280 to ESP32)
- **SCK:** Serial Clock signal
- **CS:** Chip Select pin used to enable SPI communication

ADC Functionality:

- The BMP280 has an internal ADC used for converting analog pressure and temperature signals into digital values.
- This eliminates the need for an external ADC.

Power Pins:

- VCC supplies power to the sensor.
- GND provides the reference ground.

Communication Interface:

- Supports both I2C and SPI communication protocols.
- I2C is commonly used for simple connections with ESP32.

Specifications

- **Sensor Type:** Digital pressure and temperature sensor
 - **Pressure Range:** 300 hPa to 1100 hPa
 - **Temperature Range:** -40°C to +85°C
 - **Pressure Accuracy:** ± 1 hPa
 - **Temperature Accuracy:** ± 1 °C
 - **Resolution:**
 - **Pressure:** Up to 0.16 Pa
 - **Temperature:** 0.01°C
 - **Interface:** I2C / SPI
- Operating Voltage:** 1.8V to 3.6V
- Power Consumption:** Very low power operation

4.4 MPU6050 IMU SENSOR

A figure 6 Represents MPU6050 IMU Sensor, which is a widely used motion tracking device that combines a 3-axis accelerometer and a 3-axis gyroscope in a single chip. It is used to measure acceleration, angular velocity, and orientation of a system. The MPU6050 is commonly used in embedded systems, robotics, drones, and aerospace applications.

The sensor provides real-time motion data and is highly suitable for applications requiring stability, orientation detection, and motion analysis. It communicates with microcontrollers like ESP32 using the I2C protocol.

The MPU6050 also includes a Digital Motion Processor (DMP) that can process complex motion algorithms internally, reducing the computational load on the microcontroller.

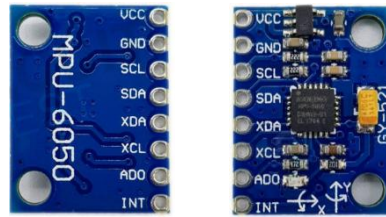


Figure 6: MPU6050 IMU Sensor

Pin Description

- **VCC:** Power supply input (3.3V to 5V)
- **GND:** Ground connection
- **SDA:** Serial Data line for I2C communication
- **SCL:** Serial Clock line for I2C communication
- **XDA:** Auxiliary I2C data (for external sensors)
- **XCL:** Auxiliary I2C clock
- **AD0:** I2C address selection pin
- **INT:** Interrupt output pin

I2C Pins (SDA and SCL):

- **SDA (Serial Data):** Used for transmitting and receiving data between MPU6050 and ESP32.
- **SCL (Serial Clock):** Used to synchronize communication between devices.

Interrupt Pin:

- **INT:** Generates interrupt signals when new sensor data is available.
- Helps in efficient data reading without continuous polling.

Sensor Components:

- **3-axis Accelerometer:** Measures linear acceleration (X, Y, Z axes)
- **3-axis Gyroscope:** Measures angular velocity (rotation along X, Y, Z axes)

ADC Functionality:

- The MPU6050 contains internal ADCs to convert analog signals from sensors into digital data.
- Provides processed digital output directly to the microcontroller.

Power Pins:

- VCC supplies power to the sensor module.
- GND provides the reference ground.

Communication Interface:

- Uses I2C communication protocol.
- Default I2C address: 0x68 (can be changed to 0x69 using AD0 pin).

Specifications

- Sensor Type: 6-axis IMU (3-axis accelerometer + 3-axis gyroscope)

Accelerometer Range:

- $\pm 2g, \pm 4g, \pm 8g, \pm 16g$

Gyroscope Range:

- $\pm 250, \pm 500, \pm 1000, \pm 2000$ °/s

- **Operating Voltage:** 3.3V to 5V
- **Communication Protocol:** I2C
- Built-in DMP (Digital Motion Processor)
- Low power consumption

4.5 NEO6M GPS MODULE

The Figure 7 Represents GPS module is a device that allows you to receive signals from GPS satellites and determine your precise location on Earth.

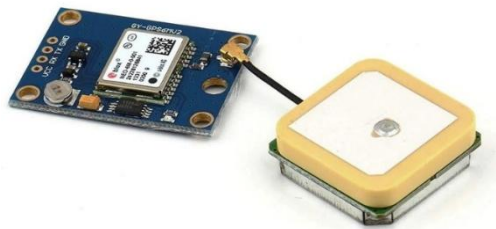


Figure 7: NEO6M GPS Module

Satellite Communication: GPS satellites orbit the Earth and continuously transmit signals that include information about the satellite's position and the current time. A GPS module receives these signals and uses the information to calculate its own position.

Triangulation: The GPS module receives signals from multiple satellites. By analyzing the time it takes for the signals to travel from the satellites to the module, the module can calculate the distance to each satellite. With distances from at least four satellites, the module can determine its precise location using trilateration algorithms.

Position and Timing: In addition to determining its own latitude, longitude, and altitude, a GPS module can also provide accurate timing information. This timing data is often used in applications that require precise synchronization.

Communication Interfaces: GPS modules typically have communication interfaces that allow them to connect to external devices such as microcontrollers, computers, or smartphones. Common communication protocols include UART (serial communication) and I2C.

NMEA Sentences: Like the NEO-6M module mentioned earlier, GPS modules commonly output data in the form of NMEA sentences. These sentences contain information about the GPS fix status, latitude, longitude, altitude, speed, and more. NMEA sentences are ASCII strings that can be parsed easily.

Antenna: GPS modules require an external antenna to receive signals from satellites. The quality and placement of the antenna can impact the accuracy of the module's positioning.

Pin Description

VCC: This pin is used to provide power to the GPS module. It is usually connected to a positive supply voltage, such as 3.3V or 5V, depending on the module specifications.

GND: This pin is connected to the ground or common reference of the power supply.

TX or Transmit Data: This pin is used for serial communication and sends data from the GPS module to an external device, like a microcontroller or computer.

RX or Receive Data: This pin is also used for serial communication and receives data from an external device.

Specifications

- **Receiver Type:** 50-channel u-blox 6 receiver
- **Update Rate:** Up to 5 Hz (5 times per second)
- **Interface:** UART
- **Antenna:** External
- **Supply Voltage:** 3.3V
- **Power Consumption:** 30 mA
- **Operating Temperature:** -40°C to +85°C
- **Dimensions:** 22.4 mm × 17 mm
- **Dynamics:** Up to 4g acceleration, up to 515 m/s velocity
- **Time Pulse:** 1 pulse per second (PPS) output

4.6 NRF24L01 LONG RANGE MODULE

The Figure 8 Represents NRF24L01 Module is a low-power wireless communication module designed for short to medium-range data transmission using radio frequency (RF). It operates in the 2.4 GHz ISM band and is widely used in embedded systems for wireless communication between microcontrollers.



Figure 8: NRF24L01 Long Range Module

The NRF24L01 module enables reliable and high-speed data transmission with low latency. It is commonly used in applications such as remote-control systems, telemetry, wireless sensor networks, and IoT-based communication systems. In the IRIS Aerospace

Rocket System, the NRF24L01 module is used for bidirectional telemetry communication between the rocket (transmitter) and the Ground Control Station (receiver).

Wireless Communication: The NRF24L01 uses radio frequency signals to transmit and receive data between two devices. It supports multiple channels and can communicate with multiple nodes in a network.

Data Transmission: The module supports data rates of 250 kbps, 1 Mbps, and 2 Mbps. It uses packet-based communication with built-in error detection and retransmission mechanisms.

Addressing: The NRF24L01 supports multiple communication pipes (up to 6), allowing communication with multiple devices using unique addresses.

Power Efficiency: The module is designed for low power consumption and supports power-saving modes such as standby and power-down modes.

Pin Description

VCC: This pin is used to supply power to the NRF24L01 module. It operates typically at 3.3V (must not exceed 3.6V).

GND (Ground): This pin is connected to the ground reference.

CE (Chip Enable): This pin is used to activate the module for transmission or reception.

CSN (Chip Select Not): This pin is used for SPI communication to select the device.

SCK (Serial Clock): This pin is used to provide clock signal for SPI communication.

MOSI (Master Out Slave In): This pin is used to send data from microcontroller to NRF24L01.

MISO (Master In Slave Out): This pin is used to receive data from NRF24L01 to microcontroller.

IRQ (Interrupt Request): This pin is used to notify the microcontroller about transmission or reception events.

Communication Interface

- **SPI Protocol:** The NRF24L01 communicates with microcontrollers using SPI (Serial Peripheral Interface). It requires MOSI, MISO, SCK, and CSN pins for communication.

Operating Modes

- **Transmit Mode:** Used to send data to another NRF24L01 module.
- **Receive Mode:** Used to receive incoming data.
- **Standby Mode:** Low power mode when module is inactive.
- **Power Down Mode:** Minimum power consumption mode.

Specifications

- **Frequency Band:** 2.4 GHz ISM band
- **Data Rate:** 250 kbps, 1 Mbps, 2 Mbps
- **Operating Voltage:** 1.9V to 3.6V (Typical: 3.3V)
- **Communication:** SPI
- **Range:** Up to 100 meters (standard), up to 1 km with external antenna (long-range version)
- **Power Consumption:** Low power operation
- **Output Power:** Programmable (-18 dBm to 0 dBm)
- **Channels:** 125 selectable channels
- **Multi-device Support:** Up to 6 data pipes

4.7 SG90 SERVO MOTOR

The SG90 servo motor is a small and lightweight actuator used for precise position control in embedded systems and robotics applications. It is widely used in projects that require controlled angular movement such as robotic arms, control surfaces, and stabilization systems.

The SG90 servo motor operates based on Pulse Width Modulation (PWM) signals provided by a microcontroller such as ESP32. It can rotate within a limited angular range, typically from 0° to 180°, allowing accurate positioning. In the IRIS Aerospace Rocket System, the SG90 servo motors are used for fin control and thrust vector control (TVC) to maintain stability during flight.



Figure 9: SG90 Servo Motor

Working Principle: The servo motor consists of a DC motor, control circuit, and position feedback system. It continuously compares the desired position (input PWM signal) with the current position and adjusts the motor accordingly.

PWM Control: The servo position is controlled using PWM signals. A pulse width of approximately 1ms corresponds to 0°, 1.5ms to 90°, and 2ms to 180°.

Pin Description

- **VCC:** This pin is used to supply power to the servo motor (typically 5V).
- **GND:** This pin is connected to the ground reference.
- **Signal:** This pin receives PWM signals from the microcontroller to control the servo position.

Control Signal (PWM):

- The SG90 servo is controlled using PWM signals
- Pulse width determines angular position
- **Typical frequency:** 50 Hz

Operating Range

- **Rotation Range:** 0° to 180°
- Position controlled within limited angular range

Specifications

- **Motor Type:** Micro Servo Motor
- **Operating Voltage:** 4.8V to 6V
- **Torque:** 1.8 kg-cm (approx.)
- **Speed:** 0.1 sec/60°

- **Control Signal:** PWM
- **Weight:** ~9 grams
- **Rotation:** 0° to 180°

4.9 MP1584 BUCK CONVERTER

The MP1584 Buck Converter is a high-efficiency DC-DC step-down voltage regulator used to convert a higher input voltage into a lower stable output voltage. It is widely used in embedded systems and electronics projects to provide a regulated power supply to components such as microcontrollers, sensors, and communication modules.

The MP1584 operates using switching regulator technology, which ensures high efficiency and low heat generation compared to linear regulators. It can deliver a constant output voltage even when the input voltage fluctuates.

Working Principle

The buck converter reduces voltage by rapidly switching the input voltage on and off and using inductors and capacitors to smooth the output. This process ensures efficient power conversion with minimal energy loss.



Figure 10: MP1584 Buck Converter

Pin Description

- **VIN+:** Input positive voltage supply
- **VIN- (GND):** Input ground
- **VOUT+:** Regulated output voltage
- **VOUT- (GND):** Output ground

Adjustment Control

- **Potentiometer:** Used to adjust the output voltage
- Clockwise rotation increases voltage
- Counterclockwise rotation decreases voltage

Specifications

- **Module Type:** DC-DC Buck Converter
- **Input Voltage:** 4.5V to 28V
- **Output Voltage:** 0.8V to 20V (adjustable)
- **Output Current:** Up to 3A (max)
- **Efficiency:** Up to 90%
- **Switching Frequency:** 1.5 MHz
- **Voltage Regulation:** High stability
- **Protection:** Thermal shutdown and current limiting

4.10 LITHIUM-ION BATTERY

The Lithium-Ion (Li-ion) battery is a rechargeable energy storage device widely used in portable electronic systems due to its high energy density, lightweight design, and long cycle life. It provides a stable and reliable power source for various electronic components and systems.

Lithium-ion batteries operate based on the movement of lithium ions between the anode and cathode during charging and discharging cycles. They are preferred in embedded and aerospace applications due to their compact size, efficiency, and ability to deliver consistent voltage output.



Figure 10: Lithium-ion Battery

Working Principle

Lithium-ion batteries store energy through electrochemical reactions. During discharge, lithium ions move from the anode to the cathode, generating electric current. During charging, the process is reversed, restoring the battery's energy.

Pin Description

- **Positive Terminal (+):** Supplies positive voltage to the circuit
- **Negative Terminal (-):** Connected to ground reference

Battery Characteristics

- Rechargeable battery
- High energy density
- Low self-discharge rate
- Long life cycle

Specifications

- **Battery Type:** Lithium-Ion (Li-ion)
- **Nominal Voltage:** 3.7V
- **Fully Charged Voltage:** 4.2V
- **Capacity:** Typically 2200 mAh to 3000 mAh (varies by model)
- **Discharge Current:** Depends on battery rating
- **Charging Method:** Constant Current / Constant Voltage (CC/CV)
- **Cycle Life:** 300–500 charge cycles

CHAPTER 5

SOFTWARE

5.1 ARDUINO IDE

The Arduino IDE, also known as Arduino Software, provides a comprehensive environment for developing and programming ESP32-based projects. It is widely used due to its simplicity, flexibility, and support for multiple hardware platforms. The Arduino IDE allows users to write, compile, and upload code to microcontrollers efficiently.

1.Text Editor:

The IDE includes a built-in text editor where users can write and edit their code. It supports syntax highlighting, automatic indentation, and error detection, making it easier to develop programs. Users can create, modify, and organize Arduino sketches (programs) in this editor.

2. Message Area:

The message area displays important information such as compilation status, warnings, and error messages. It helps users identify issues in their code during development and debugging processes.

3. Toolbar and Buttons:

The toolbar contains buttons for common functions such as verifying (compiling) code, uploading code to the ESP32 board, opening files, and saving projects. These buttons streamline workflow and provide quick access to essential actions.

4. Menus:

The IDE features various menus such as File, Edit, Sketch, Tools, and Help. These menus provide additional options including managing libraries, configuring preferences, selecting boards, and setting communication ports.

5. Hardware Interaction:

The Arduino IDE connects seamlessly with ESP32 hardware. Users can upload compiled code (sketch) to the ESP32 board via USB connection, enabling execution and real-time interaction with the system.

UPLOAD

The upload button compiles and runs the written code and uploads it to the connected ESP32 board. Before uploading, the correct board and port must be selected. Once the USB is connected to the computer, the port appears in the IDE. After successful upload, the program starts executing. If any error occurs, it will be displayed in the message area.

OPEN BUTTON

The open button is used to open files that has already been created and saved. The selected file will be opened in the current window.

SAVE BUTTON

This button saves the code.

NEW BUTTON

The new button creates a new display window to create a new code.

VERIFY BUTTON

The verify button works the same as the compiler window in any other programming IDE's it compiles the code and shows error if any.

SERIAL MONITOR

The serial monitor is a display window which can be used for debugging, testing concepts or to communicate directly with the ESP32 board. It allows the exchange of data with the connected board on the port.

PREPARING YOUR ARDUINO IDE FOR ESP32

1. First, make sure you have the latest version of Arduino IDE:
<https://www.arduino.cc/en/Main/Software>
2. Then we need to add the ESP32 to the Boards list. Open the Arduino IDE Preferences panel.
On Windows: **File** → **Preferences**
On Mac: **Arduino** → **Preferences**

3. Add the following URL to the “Additional Boards Manager URLs”:

https://dl.espressif.com/dl/package_esp32_index.json as shown in Figure 11.

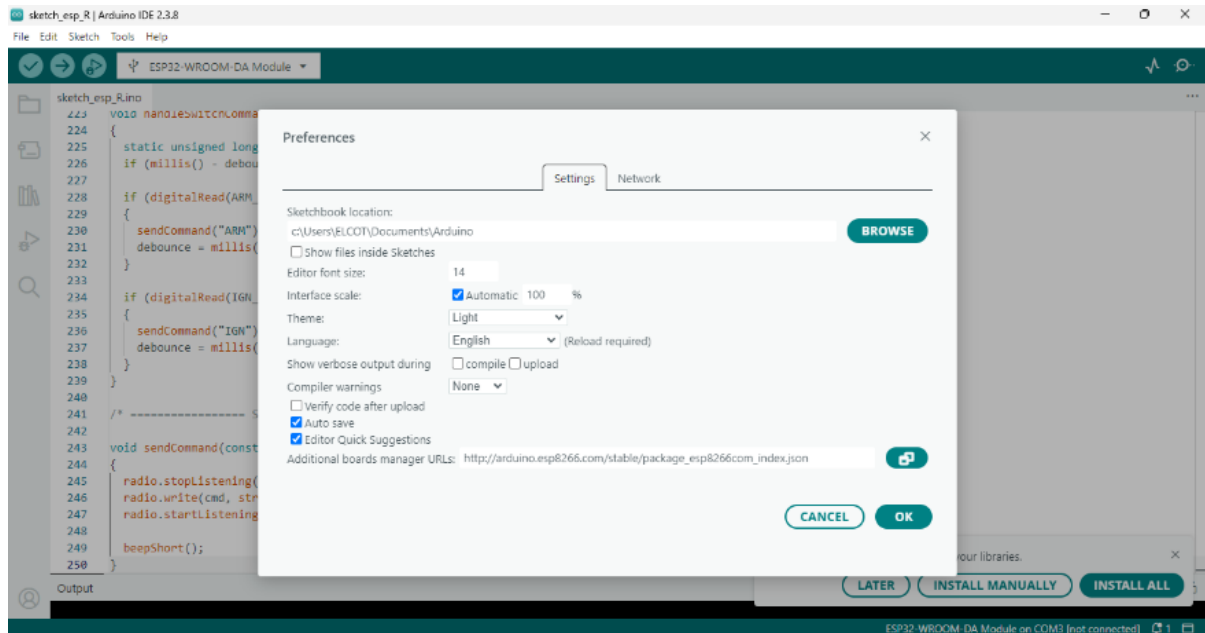


Figure 11: Additional Boards Manager URLs

- Open Arduino Boards Manager "Tools->Board->Boards Manager..." as shown Fig 12
- And search for "ESP32-WROOM-DA Module" and click "Install."
- After install, now we can see the "ESP32-WROOM-DA Module" sub-menu under Boards

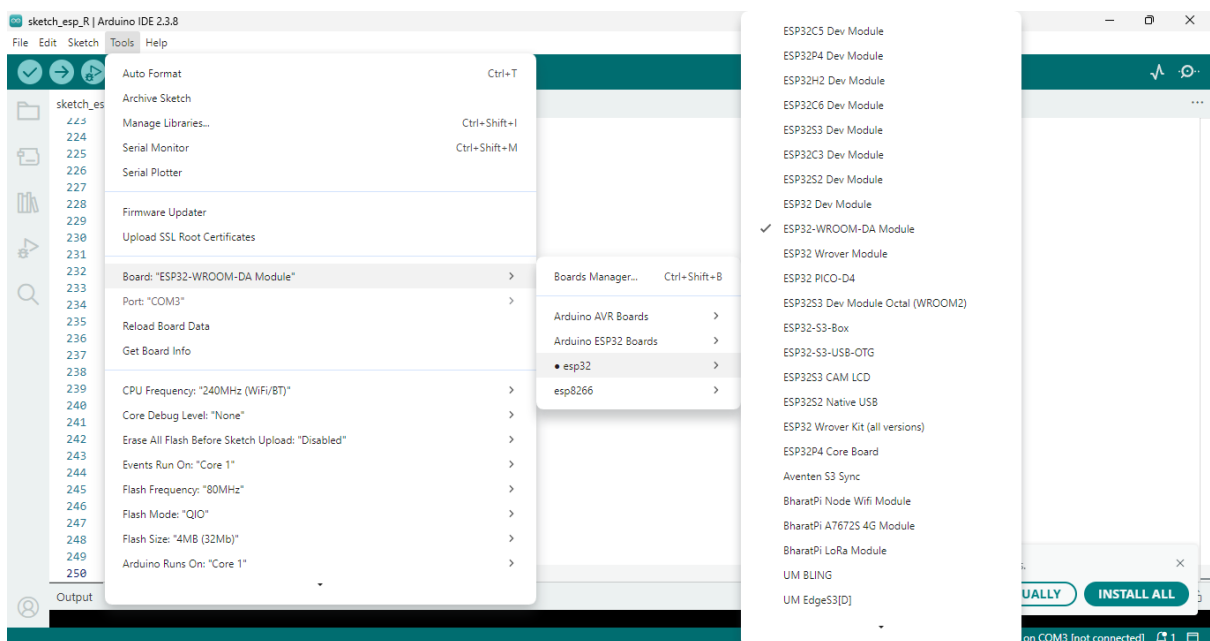


Figure 12: Additional Boards Manager URLs

- Use the screen to program after select the ESP32 Dev Module
- And check the working of program with compiler
- And fuse the program to ESP32 Dev Module board

5.2 COSMIC TOWER

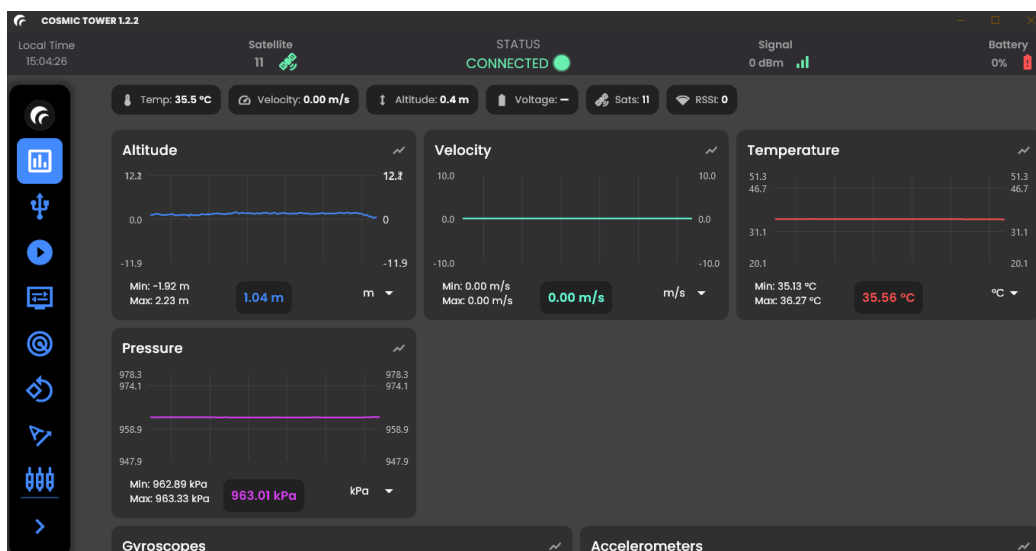
The Cosmic Tower is an advanced Ground Control Software developed by Cosmic Aerospace for real-time telemetry monitoring, data visualization, and command control of aerospace systems. It is specifically designed to support experimental rocketry by providing a reliable interface between the onboard flight computer and the ground station.

The software enables continuous acquisition, decoding, and visualization of telemetry data transmitted from the rocket. It ensures efficient monitoring of critical flight parameters such as altitude, pressure, temperature, acceleration, and orientation. Cosmic Tower enhances mission safety and performance analysis by offering a structured and user-friendly platform.

Working Principle:

The onboard flight computer (ESP32) collects sensor data and encodes it into a structured telemetry format. This data is transmitted wirelessly via the NRF24L01 communication module. The ground station receiver (NodeMCU CP2102) receives the data and forwards it to the Cosmic Tower software through serial communication.

The Cosmic Tower software decodes the telemetry packets, maps the parameters, and displays them in real-time using graphical interfaces.



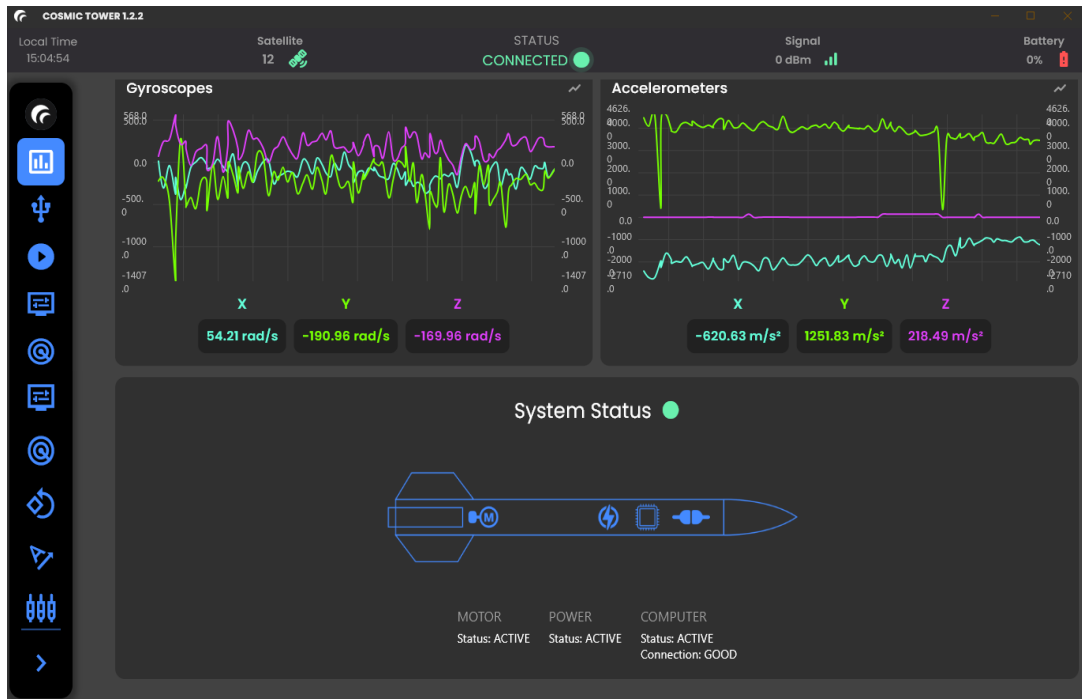


Figure 13: Telemetry Data

Telemetry System:

- Real-time telemetry acquisition
- Multi-sensor data integration
- Structured packet-based communication
- High-speed wireless transmission

Telemetry Format:

- ASCII-based data protocol
- Frame starts with '@' and ends with ''
- Single-letter parameter identifiers
- Multi-rate sensor data transmission
- Includes system status block

Data Parameters:

- Temperature
- Pressure
- Altitude
- Voltage
- GPS Latitude and Longitude
- Gyroscope (X, Y, Z)
- Accelerometer (X, Y, Z)
- System Status / Redundancy Data

Communication System:

- Transmitter: ESP32 Flight Computer + NRF24L01
- Receiver: NodeMCU CP2102 + NRF24L01
- Communication Type: Wireless RF Communication
- Interface: SPI (NRF24) and UART (Serial Output)

Ground Station Functions:

- Real-time telemetry visualization
- Graphical plotting of flight parameters
- Data logging and storage
- System health monitoring
- Command transmission to rocket

Command Functions:

- ARM
- DISARM
- RESET
- PARACHUTE DEPLOYMENT
- IGNITION CONTROL

Software Features:

- User-friendly graphical interface
- Real-time plotting using MATLAB Simulink
- High-speed data processing
- Reliable packet decoding system

Safety Features:

- Telemetry link monitoring
- Automatic alert on signal loss
- Emergency command execution
- Redundant status monitoring

5.3 FLOW CHART

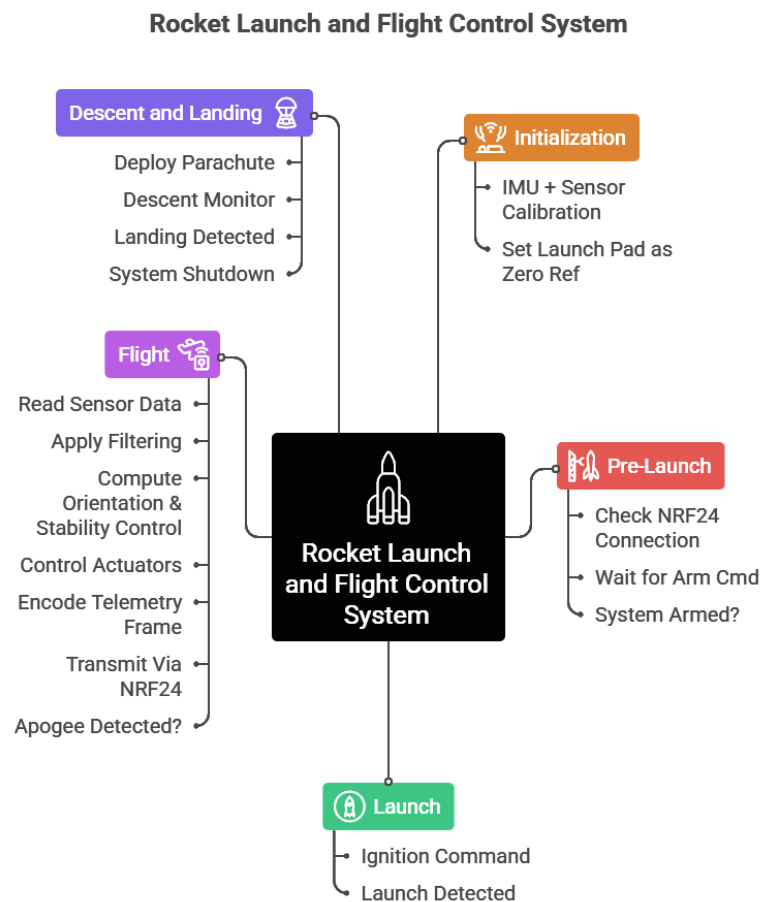


Figure 14: Flow Chart

5.4 PROGRAM

IRIS Transmitter

```
#include <SPI.h>
#include <WiFi.h>
#include <MPU6050.h>
#include <Adafruit_BMP280.h>
#include <TinyGPSPlus.h>
#include <RF24.h>
#include <ESP32Servo.h>
#include "esp_wifi.h"
#include "esp_bt.h"

-----NRF ADDRESS ----- */
const uint8_t address[6] = "IRIS1";
/* ----- LIMITS ----- */
#define SERVO_CENTER 90
#define FIN_LIMIT 15
#define TVC_LIMIT 10
#define SERVO_RATE_LIMIT 2
#define GYRO_DEADBAND 5
#define LPF_ALPHA 0.2
/* ----- PINS ----- */
#define SDA_PIN 21
#define SCL_PIN 22
#define GPS_RX 16
#define GPS_TX 17

#define NRF_CE 4
#define NRF_CSN 5
#define NRF_MOSI 23
#define NRF_MISO 19
#define NRF_SCK 18

#define FIN1 25
```

```

#define FIN2 26
#define FIN3 27
#define FIN4 14
#define TVC_PITCH 32
#define TVC_YAW 33

#define BUZZER_PIN 13

/* ----- OBJECTS ----- */
MPU6050 mpu;
Adafruit_BMP280 bmp;
TinyGPSPlus gps;
RF24 radio(NRF_CE, NRF_CSN);

Servo fin1, fin2, fin3, fin4, tvcP, tvcY;

/*-----VARIABLES ----- */
float baseAltitude = 0;
float altitude = 0;
float prevAltitude = 0;
float velocity = 0;
float temperature = 0;
float pressure = 0;

int16_t ax, ay, az;
int16_t gx, gy, gz;

float gyroOffsetX = 0;
float gyroOffsetY = 0;
float gyroOffsetZ = 0;
float filteredGX = 0;
float filteredGY = 0;

int currentFin1 = SERVO_CENTER;
int currentFin2 = SERVO_CENTER;
int currentFin3 = SERVO_CENTER;

```

```

int currentFin4 = SERVO_CENTER;
int currentTvcP = SERVO_CENTER;
int currentTvcY = SERVO_CENTER;

double latitude = 0;
double longitude = 0;
int satellites = 0;
unsigned long lastIMU=0,lastBMP=0,lastVel=0,lastTX=0,lastControl=0;

/* -----DISABLE WIFI+BT -----*/
void disableWireless(){
  WiFi.mode(WIFI_OFF);
  btStop();
  esp_wifi_stop();
  esp_bt_controller_disable();
}
/* ----- BOOT BUZZER -----*/
void playBootTone()
{
  // Generate ~2kHz tone manually
  for (int i = 0; i < 400; i++) // duration control
  {
    digitalWrite(BUZZER_PIN, HIGH);
    delayMicroseconds(250); // 250us HIGH
    digitalWrite(BUZZER_PIN, LOW);
    delayMicroseconds(250); // 250us LOW
  }
}
/* -----IMU CALIBRATION -----*/
void calibrateIMU(){
  Serial.println("Calibrating IMU...");
  long sumX=0,sumY=0,sumZ=0;

  for(int i=0;i<1000;i++){

```

```

    mpu.getMotion6(&ax,&ay,&az,&gx,&gy,&gz);
    sumX+=gx; sumY+=gy; sumZ+=gz;
    delay(2);
}

gyroOffsetX=sumX/1000.0;
gyroOffsetY=sumY/1000.0;
gyroOffsetZ=sumZ/1000.0;

Serial.println("IMU Calibration Complete");
}
/* -----SETUP -----*/
void setup(){
    disableWireless();
    Serial.begin(115200);
    Serial2.begin(9600,SERIAL_8N1,GPS_RX,GPS_TX);

    Wire.begin(SDA_PIN,SCL_PIN);
    SPI.begin(NRF_SCK,NRF_MISO,NRF_MOSI,NRF_CSN);

    pinMode(BUZZER_PIN, OUTPUT);
    playBootTone(); // Boot indicator sound
    mpu.initialize();
    bmp.begin(0x76);
    delay(2000);
    calibrateIMU();
    baseAltitude = bmp.readAltitude(1013.25);

    fin1.attach(FIN1);
    fin2.attach(FIN2);
    fin3.attach(FIN3);
    fin4.attach(FIN4);
    tvcP.attach(TVC_PITCH);
    tvcY.attach(TVC_YAW);

```

```

fin1.write(SERVO_CENTER);
fin2.write(SERVO_CENTER);
fin3.write(SERVO_CENTER);
fin4.write(SERVO_CENTER);

radio.begin();
radio.enableDynamicPayloads();
radio.setPALevel(RF24_PA_MAX);
radio.setDataRate(RF24_250KBPS);
radio.setChannel(108);
radio.setAutoAck(false);
radio.openWritingPipe(address);
radio.stopListening();

Serial.println("IRIS FLIGHT COMPUTER READY");
}
/* ----- LOOP ----- */
void loop(){
  unsigned long now=millis();

  /* ===== IMU 200Hz ===== */
  if(now-lastIMU>=5){
    lastIMU=now;
    mpu.getMotion6(&ax,&ay,&az,&gx,&gy,&gz);

    gx-=gyroOffsetX;
    gy-=gyroOffsetY;
    gz-=gyroOffsetZ;
  }

  /* ----- BMP 20Hz ----- */
  if(now-lastBMP>=50){
    lastBMP=now;
    temperature=bmp.readTemperature();
  }
}

```

```

pressure=bmp.readPressure()/100.0;
altitude=bmp.readAltitude(1013.25)-baseAltitude;
}

/* ----- Velocity ----- */
if(now-lastVel>=50){
    float dt=(now-lastVel)/1000.0;
    velocity=(altitude-prevAltitude)/dt;
    prevAltitude=altitude;
    lastVel=now;
}

/* -----Stabilization 50Hz-----*/
if(now-lastControl>=20){
    lastControl=now;
    stabilize();
}

/* -----GPS -----*/
while(Serial2.available()){
    gps.encode(Serial2.read());
    if(gps.location.isUpdated()){
        latitude=gps.location.lat();
        longitude=gps.location.lng();
        satellites=gps.satellites.value();
    }
}

/* -----Telemetry 20Hz ----- */
if(now-lastTX>=50){
    lastTX=now;
    sendTelemetry();
}
}

```

```

/*----- STABILIZATION ----- */
void stabilize()
{
    /* ===== LOW PASS FILTER ===== */
    filteredGX = (LPF_ALPHA * gx) + ((1 - LPF_ALPHA) * filteredGX);
    filteredGY = (LPF_ALPHA * gy) + ((1 - LPF_ALPHA) * filteredGY);

    /* ===== DEAD BAND ===== */
    if (abs(filteredGX) < GYRO_DEADBAND) filteredGX = 0;
    if (abs(filteredGY) < GYRO_DEADBAND) filteredGY = 0;

    /* ----- TVC CONTROL----- */
    int pitchTvcTarget = constrain(-filteredGX / 60, -TVC_LIMIT, TVC_LIMIT);
    int yawTvcTarget  = constrain(-filteredGY / 60, -TVC_LIMIT, TVC_LIMIT);

    int targetTvcP = SERVO_CENTER + pitchTvcTarget;
    int targetTvcY = SERVO_CENTER + yawTvcTarget;

    currentTvcP += constrain(targetTvcP - currentTvcP, -SERVO_RATE_LIMIT,
SERVO_RATE_LIMIT);
    currentTvcY += constrain(targetTvcY - currentTvcY, -SERVO_RATE_LIMIT,
SERVO_RATE_LIMIT);

    tvcP.write(currentTvcP);
    tvcY.write(currentTvcY);

    /* ----- FIN CONTROL ----- */
    int pitchFinTarget = constrain(-filteredGX / 60, -FIN_LIMIT, FIN_LIMIT);
    int yawFinTarget  = constrain(-filteredGY / 60, -FIN_LIMIT, FIN_LIMIT);

    int targetFin1 = SERVO_CENTER + pitchFinTarget;
    int targetFin2 = SERVO_CENTER - pitchFinTarget;
    int targetFin3 = SERVO_CENTER + yawFinTarget;
    int targetFin4 = SERVO_CENTER - yawFinTarget;

```



```

    currentFin1 += constrain(targetFin1 - currentFin1, -SERVO_RATE_LIMIT,
SERVO_RATE_LIMIT);
    currentFin2 += constrain(targetFin2 - currentFin2, -SERVO_RATE_LIMIT,
SERVO_RATE_LIMIT);
    currentFin3 += constrain(targetFin3 - currentFin3, -SERVO_RATE_LIMIT,
SERVO_RATE_LIMIT);
    currentFin4 += constrain(targetFin4 - currentFin4, -SERVO_RATE_LIMIT,
SERVO_RATE_LIMIT);

    fin1.write(currentFin1);
    fin2.write(currentFin2);
    fin3.write(currentFin3);
    fin4.write(currentFin4);
}
/*-----TELEMETRY ----- */
void sendTelemetry()
{
    char part1[32];
    char part2[32];
    char part3[48];

    snprintf(part1, sizeof(part1),
        "@1A%.2fT%.2fP%.2fV%.2f",
        altitude,
        temperature,
        pressure,
        12.50);

    radio.write(part1, strlen(part1) + 1);
    delay(5);

    snprintf(part2, sizeof(part2),
        "@2x%d y%d z%d X%d Y%d Z%d",
        gx, gy, gz,

```

```

        ax, ay, az);

radio.write(part2, strlen(part2) + 1);
delay(5);

snprintf(part3, sizeof(part3),
        "@3I%.6fL%.6fS%d\\",
        latitude,
        longitude,
        satellites);

radio.write(part3, strlen(part3) + 1); }

```

// IRIS Receiver code

```

#include <SPI.h>
#include <RF24.h>
#include <ESP8266WiFi.h>
/* ----- NRF CONFIG ----- */
#define NRF_CE D2
#define NRF_CSN D8
/*-----IO PINS -----*/
#define LED_PIN D0 // GREEN LED (connected)
#define RED_LED 3 // RX pin for RED LED
#define ARM_SW D3
#define IGN_SW D4
#define BUZZER_PIN D1

const uint8_t address[6] = "IRIS1";
RF24 radio(NRF_CE, NRF_CSN);

/* ----- TELEMETRY BUFFERS ----- */

```

```

char part1[64] = "";
char part2[64] = "";
char part3[64] = "";

unsigned long lastPacketTime = 0;
unsigned long icuTimer = 0;

bool firstConnectionMade = false;
bool linkLost = false;
/*-----BASIC BUZZER ----- */

void beepShort()
{
    tone(BUZZER_PIN, 2200);
    delay(80);
    noTone(BUZZER_PIN);
}

void beepLong()
{
    tone(BUZZER_PIN, 1200);
    delay(300);
    noTone(BUZZER_PIN);
}

void bootBeep()
{
    beepShort();
    delay(80);
    beepShort();
}

```

```
/* ----- ROCKET CONNECTION SOUND-----*/
```

```
void rocketConnectSound()
{
  for (int f = 1200; f <= 3200; f += 160)
  {
    tone(BUZZER_PIN, f);
    delay(40);
  }
  noTone(BUZZER_PIN);
}
```

```
/* -----SIGNAL LOST SOUND ----- */
```

```
void signalLostSound()
{
  for (int f = 2200; f >= 700; f -= 140)
  {
    tone(BUZZER_PIN, f);
    delay(70);
  }
  noTone(BUZZER_PIN);
}
```

```
/* ----- RECONNECT SOUND -----*/
```

```
void reconnectSound()
{
  tone(BUZZER_PIN, 1800); delay(120);
  tone(BUZZER_PIN, 2500); delay(120);
  tone(BUZZER_PIN, 3200); delay(150);
  noTone(BUZZER_PIN);
}
```

```

void setup()
{
  WiFi.mode(WIFI_OFF);
  WiFi.forceSleepBegin();
  delay(1);

  Serial.begin(115200);

  pinMode(LED_PIN, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(ARM_SW, INPUT_PULLUP);
  pinMode(IGN_SW, INPUT_PULLUP);
  pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(LED_PIN, LOW);
  digitalWrite(RED_LED, HIGH); // RED ON before connection
  digitalWrite(BUZZER_PIN, LOW);

  bootBeep();
  SPI.begin();
  if (!radio.begin())
  {
    Serial.println("NRF FAILED");
    while (1);
  }

  radio.enableDynamicPayloads();
  radio.setPALevel(RF24_PA_MAX);
  radio.setDataRate(RF24_250KBPS);
  radio.setChannel(108);
  radio.setAutoAck(false);

```

```

radio.openReadingPipe(0, address);
radio.startListening();

Serial.println("GCS READY");
}

void loop()
{
    receiveTelemetry();
    handleSwitchCommands();
}

//-----RECEIVE TELEMETRY -----

void receiveTelemetry()
{
    if (radio.available())
    {
        uint8_t len = radio.getDynamicPayloadSize();

        if (len > 0 && len <= 32)
        {
            char buffer[33] = {0};
            radio.read(buffer, len);
            buffer[len] = '\0';

            lastPacketTime = millis();

            /* ----- PACKET 1 ----- */
            if (strncmp(buffer, "@1", 2) == 0)
                strncpy(part1, buffer + 2, sizeof(part1) - 1);

            /* ----- PACKET 2 ----- */
            else if (strncmp(buffer, "@2", 2) == 0)
                strncpy(part2, buffer + 2, sizeof(part2) - 1);
        }
    }
}

```

```

/* ----- PACKET 3 ----- */
else if (strcmp(buffer, "@3", 2) == 0)
{
    strncpy(part3, buffer + 2, sizeof(part3) - 1);

    if (strchr(part3, "") != NULL)
    {
        Serial.print("@");
        Serial.print(part1);
        Serial.print(part2);
        Serial.println(part3);

        digitalWrite(LED_PIN, HIGH);
        digitalWrite(RED_LED, LOW); // RED OFF after connection

        /* FIRST CONNECTION */
        if (!firstConnectionMade)
        {
            rocketConnectSound();
            firstConnectionMade = true;
        }

        //-----LINK RESTORED-----

        else if (linkLost)
        {
            tone(BUZZER_PIN, 2500);
            delay(120);
            noTone(BUZZER_PIN);
            linkLost = false;
        }
    }
}
}
}
}

```

```

}

// ----- LINK LOST DETECT -----

if (!linkLost && millis() - lastPacketTime > 2000)
{
    digitalWrite(LED_PIN, LOW);
    digitalWrite(RED_LED, HIGH); // RED LED ON again

    signalLostSound();

    linkLost = true;
    icuTimer = millis();
}

// ----- ICU HEARTBEAT -----

if (linkLost && millis() - icuTimer > 1000)
{
    tone(BUZZER_PIN, 1500);
    delay(60);
    noTone(BUZZER_PIN);

    icuTimer = millis();
}
}

// -----COMMAND HANDLER -----

void handleSwitchCommands()
{
    static unsigned long debounce = 0;
    if (millis() - debounce < 200) return;

```



```

if (digitalRead(ARM_SW) == LOW)
{
    sendCommand("ARM");
    debounce = millis();
}

if (digitalRead(IGN_SW) == LOW)
{
    sendCommand("IGN");
    debounce = millis();
}
}

void sendCommand(const char* cmd)
{
    radio.stopListening();
    radio.write(cmd, strlen(cmd) + 1);
    radio.startListening();

    beepShort();
}

```

CHAPTER 6

TESTING

6.1 Introduction

This chapter presents the testing and validation of the IRIS Embedded Avionics and Telemetry System. The project is currently developed up to the hardware integration and communication stage, and flight testing has not been performed.

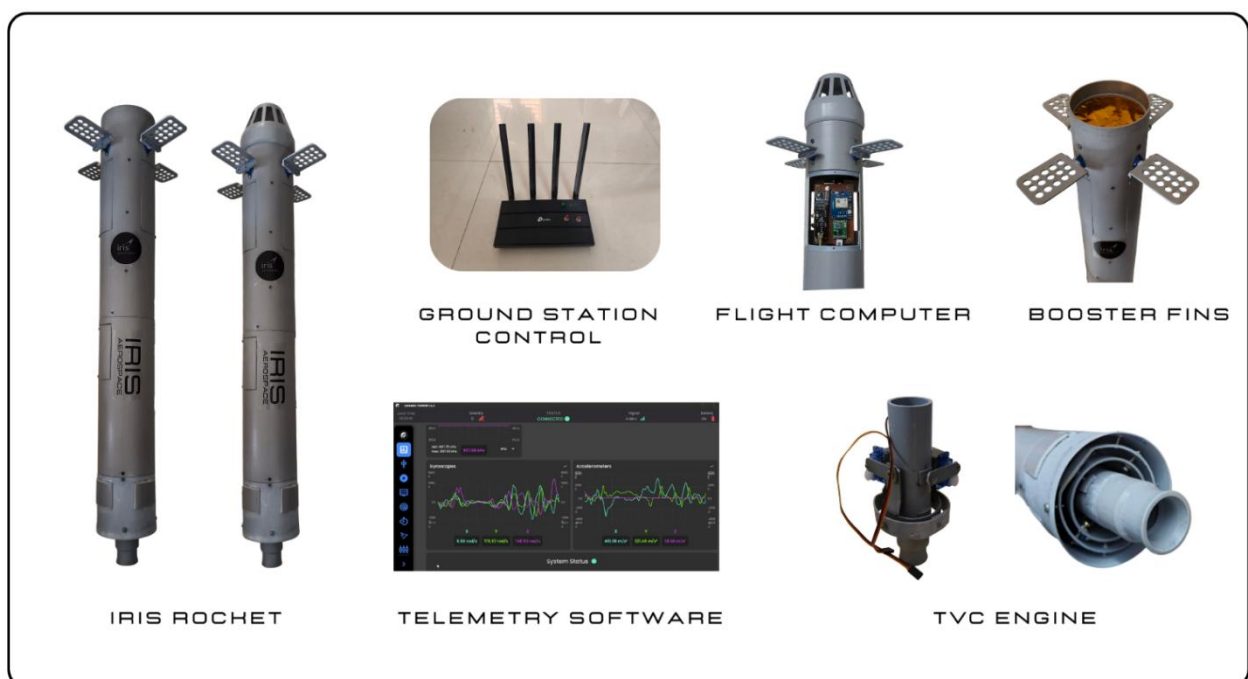


Figure 15 (a) IRIS Rocket

6.2 Booster Fins

What is Fin?

Fins are aerodynamic control surfaces attached to the rocket body.

Role in Rocket:

They help to stabilize the rocket during flight by controlling direction.

Purpose:

- Maintain balance
- Reduce unwanted rotation
- Assist in trajectory correction

Testing Procedure:

- Connected 4 servo motors to fins
- Sent angle signals from ESP32
- Tested movement left and right

Result:

- Fins moved correctly
- Smooth operation achieved
- No major vibration after tuning

6.3 TVC (Thrust Vector Control System)**What is TVC?**

TVC is a system that controls rocket direction by changing engine thrust angle.

Role in Rocket:

It provides active control during flight.

Purpose:

- Control pitch and yaw
- Improve stability
- Enable directional correction

Testing Procedure:

- Connected 2 servos (pitch & yaw)
- Sent angle commands
- Checked movement limits

Result:

- TVC working properly
- Controlled motion observed
- Angle limits maintained

6.4 Flight Computer (ESP32)**What is Flight Computer?**

ESP32 is the main controller of the rocket.

Role in Rocket:

It processes sensor data and controls all systems.

Purpose:

- Read sensors
- Control fins and TVC
- Send telemetry data

Testing Procedure:

- Uploaded program
- Checked serial output
- Verified sensor reading

Result:

- System initialized correctly
- Data processing working
- No crash observed

6.5 Sensors (MPU6050 + BMP280 + GPS)**What are Sensors?**

Devices used to measure physical parameters.

Role in Rocket:

Provide real-time flight data

Purpose:

- Measure altitude, pressure, temperature
- Detect motion and orientation
- Track location

Testing Procedure:

- Kept rocket stationary
- Performed calibration
- Observed readings

Result:

- Data received correctly
- Minor noise present
- Accuracy improved after calibration

6.6 NRF24 Telemetry Module

What is NRF24?

A wireless communication module.

Role in Rocket:

Transfers data between rocket and ground station.

Purpose:

- Send telemetry data
- Receive commands

Testing Procedure:

- Powered transmitter and receiver
- Established connection
- Sent data packets

Result:

- Communication successful
- Stable data transmission

6.7 Ground Control Station (GCS)

What is GCS?

A ground-based system used to monitor and control the rocket.

Role in Rocket System:

Acts as operator interface.

Purpose:

- Send commands (ARM, RESET, etc.)
- Monitor rocket data

Testing Procedure:

- Connected NodeMCU to PC
- Pressed control switches
- Checked response

Result:

- Commands working
- LEDs indicating status
- Safe control achieved

6.8 Telemetry Software

What is Telemetry Software?

Software used to display rocket data in real time.

Role in System:

Visualizes flight data.

Purpose:

- Graph plotting
- Data monitoring
- Data logging

Testing Procedure:

- Connected serial port
- Received telemetry data
- Observed graphs

Result:

- Real-time graphs displayed
- Data decoded correctly

6.9 Power System (Battery + MP1584)

What is Power System?

Provides required voltage to all components.

Role in Rocket:

Supplies stable power.

Purpose:

- Convert voltage
- Protect components

Testing Procedure:

- Checked voltage output
- Verified power supply

Result:

- Stable voltage observed
- No fluctuation

6.10 Conclusion

All components of the IRIS rocket were tested individually. Each system worked properly after calibration and tuning. The complete system is ready for integration and future flight testing.

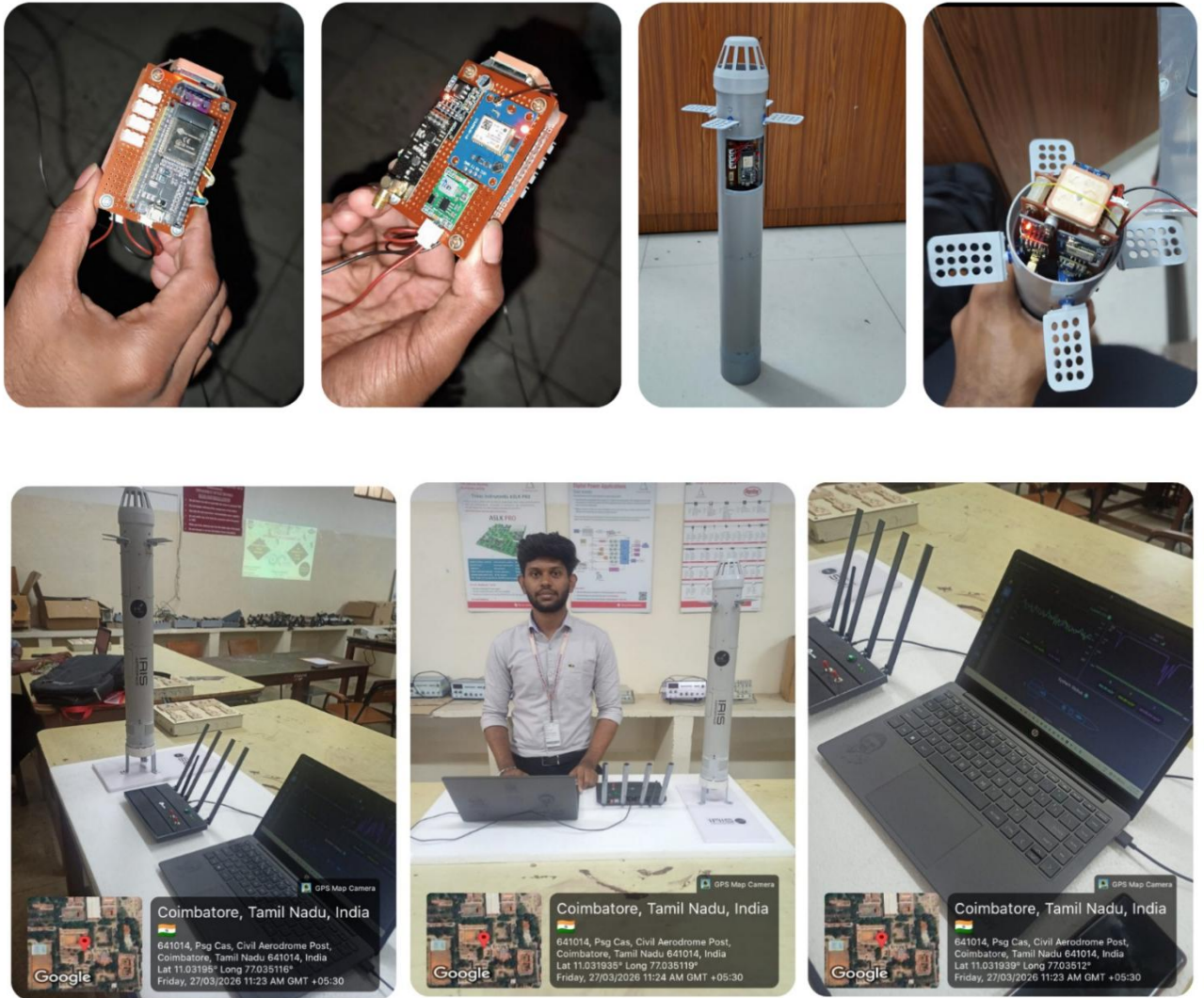


Figure 15 (b) Development of a smart embedded avionics and telemetry system for experimental rocketry (IRIS)

CHAPTER 7

CONCLUSION

7.1 CONCLUSION

The smart rocket avionics and telemetry system was successfully designed and implemented by integrating sensors, wireless communication, and control mechanisms into a unified platform. The system effectively acquires real-time flight data, including altitude, temperature, motion, and position, and transmits it to the ground control station for monitoring and analysis.

The use of ESP32, NRF24L01, and onboard sensors ensures reliable communication and accurate data acquisition, while control systems such as booster fins and thrust vector control (TVC) enhance stability and overall system performance.

In comparison with prototype rockets developed by platforms such as CSK Control Systems, this project achieves similar core functionalities, including telemetry transmission, onboard control, and stabilization. However, it emphasizes a cost-effective, modular, and student-level design approach.

Overall, the project demonstrates a compact, efficient, and scalable solution for experimental rocketry, validated through successful testing and system integration, and is suitable for further development and future flight testing.

CHAPTER 8

BIBLIOGRAPHY

References

1. Espressif Systems, “ESP32 Technical Reference Manual,” Espressif Systems, 2023.
2. Bosch Sensortec, “BMP280 Digital Pressure Sensor Datasheet,” Bosch Sensortec GmbH, 2022.
3. TDK InvenSense, “MPU6050 6-Axis Motion Tracking Device Datasheet,” TDK Corporation, 2021.
4. u-blox AG, “NEO-6M GPS Module Data Sheet,” u-blox, Switzerland, 2020.
5. Nordic Semiconductor, “nRF24L01+ Single Chip 2.4GHz Transceiver Product Specification,” Nordic Semiconductor ASA, 2021.
6. Arduino, “Arduino IDE Documentation,” Arduino.cc, 2024.
7. MathWorks, “MATLAB and Simulink for Data Acquisition and Visualization,” MathWorks Inc., 2023.
8. Monolithic Power Systems, “MP1584 Step-Down Converter Datasheet,” MPS, 2022.
9. TowerPro, “SG90 Micro Servo Motor Datasheet,” TowerPro, 2020.
10. Texas Instruments, “Designing with Lithium-Ion Batteries,” Texas Instruments Application Report, 2021.
11. Simon Monk, *Programming Arduino: Getting Started with Sketches*, McGraw-Hill Education, 2nd Edition, 2016.
12. M. H. Rashid, *Power Electronics Handbook*, Academic Press, 2017.
13. J. D. Anderson, *Introduction to Flight*, McGraw-Hill Education, 8th Edition, 2016.
14. Sutton, G. P., and Biblarz, O., *Rocket Propulsion Elements*, John Wiley & Sons, 9th Edition, 2016.

15. National Aeronautics and Space Administration (NASA), “Basics of Rocket Propulsion,” NASA Technical Reports, 2020.
16. R. Beard and T. McLain, *Small Unmanned Aircraft: Theory and Practice*, Princeton University Press, 2012.
17. P. Castillo, A. Dzul, and R. Lozano, “Real-Time Stabilization of Rocket Systems Using PID Control,” *Journal of Control Engineering*, vol. 12, no. 3, 2018.